

User and Group Management

As a system administrator, user and group management is a key skill as our users are often our main asset to manage and, usually, an organization's largest attack vector. As pentesters, understanding how to enumerate, interpret, and take advantage of users and groups is one of the easiest ways to gain access and elevate our privileges during a pentest engagement. This section will cover what users and groups are, how to manage them with PowerShell, and briefly introduce the concept of Active Directory domains and domain users.

What are User Accounts?

User accounts are a way for personnel to access and use a host's resources. In certain circumstances, the system will also utilize a specially provisioned user account to perform actions. When thinking about accounts, we typically run into four different types:

- Service Accounts
- Built-in accounts
- Local users
- Domain users

Default Local User Accounts

Several accounts are created in every instance of Windows as the OS is installed to help with host management and basic usage. Below is a list of the standard built-in accounts.

Built-In Accounts

Account	Description
Administrator	This account is used to accomplish administrative tasks on the local host.
Default Account	The default account is used by the system for running multi-user auth apps like the Xbox utility.
Guest Account	This account is a limited rights account that allows users without a normal user account to access the host. It is disabled by default and should stay that way.
WDAGUtility Account	This account is in place for the Defender Application Guard, which can sandbox application sessions.

Brief Intro to Active Directory

In a nutshell, **Active Directory** (AD) is a directory service for Windows environments that provides a central point of management for **users**, computers, **groups**, network devices, **file shares**, group policies, **devices**, and trusts with other organizations. Think of it as the gatekeeper for an enterprise environment. Anyone who is a part of the domain can access resources freely, while anyone who is not is denied access to those same resources or, at a minimum, stuck waiting in the visitors center.

Within this section, we care about AD in the context of users and groups. We can administer them from PowerShell on **any domain joined host** utilizing the **ActiveDirectory** Module. Taking a deep dive into Active Directory would take more than one section, so we will not try here. To learn more about AD, you should check out the [Introduction to Active Directory module](#).

Local vs. Domain Joined Users

Local vs. Domain Joined Users

How are they different?

Domain users differ from **local** users in that they are granted rights from the domain to access resources such as file servers, printers, intranet hosts, and other objects based on user and group membership. Domain user accounts can log in to any host in the domain, while the local user only has permission to access the specific host they were created on.

It is worth looking through the documentation on [accounts](#) to understand better how the various accounts work together on an individual Windows system and across a domain network. Take some time to look them over and understand the nuances between them. Understanding their uses and the utility of each type of account can make or break a pentesters attempt at privileged access or lateral movement during a penetration test.

What Are User Groups?

Groups are a way to sort user accounts logically and, in doing so, provide granular permissions and access to resources without having to manage each user manually. For example, we could restrict access to a specific directory or share so that only users who need access can view the files. On a singular host, this does not mean much to us. However, logical grouping is essential to maintain a proper security posture within a domain of hundreds, if not thousands, of users. From a domain perspective, we have several different types of groups that can hold not only users but end devices like PCs, printers, and even other groups. This concept is too deep of a dive for this module. However, we will talk about how to manage groups for now. If you wish to know more and get a deep dive into Active Directory and how it utilizes groups to maintain security, check out this [module](#).

Get-LocalGroup

User and Group Management	
PS C:\htb> get-localgroup	
Name	Description
----	-----
__vmware__	VMware User Group
Access Control Assistance Operators	Members of this group can remotely query authorization attributes and permissions.
Administrators	Administrators have complete and unrestricted access to the computer/domain.
Backup Operators	Backup Operators can override security restrictions for the sole purpose of backup.
Cryptographic Operators	Members are authorized to perform cryptographic operations.
Device Owners	Members of this group can change system-wide settings.
Distributed COM Users	Members are allowed to launch, activate and use Distributed COM objects on the local machine.
Event Log Readers	Members of this group can read event logs from local machine.
Guests	Guests have the same access as members of the Users group by default, except they cannot log on locally.
Hyper-V Administrators	Members of this group have complete and unrestricted access to all features of Hyper-V.
IIS_IUSRS	Built-in group used by Internet Information Services.
Network Configuration Operators	Members in this group can have some administrative privileges to manage the network.
Performance Log Users	Members of this group may schedule logging of performance counters, enable and disable logging.
Performance Monitor Users	Members of this group can access performance counter data locally and remotely.
Power Users	Power Users are included for backwards compatibility and possess limited administrative rights.
Remote Desktop Users	Members in this group are granted the right to logon remotely.
Remote Management Users	Members of this group can access WMI resources over management protocols (Distributed Management Objects).
Replicator	Supports file replication in a domain.
System Managed Accounts Group	Members of this group are managed by the system.
Users	Users are prevented from making accidental or intentional system-wide changes.

Above is an example of the local groups to a standalone host. We can see there are groups for simple things like Administrators and guest accounts, but also groups for specific roles like administrators for virtualization applications, remote users, etc. Let us interact with users and groups now that we understand them.

Adding/Removing/Editing User Accounts & Groups

Like most other things in PowerShell, we use the `get`, `new`, and `set` verbs to find, create and modify users and groups. If dealing with local users and groups, `localuser` & `localgroup` can accomplish this. For domain assets, `aduser` & `adgroup` does the trick. If we were not sure, we could always use the `Get-Command *user*` cmdlet to see what we have access to. Let us give a few of them a try.

Identifying Local Users

User and Group Management		
PS C:\htb> Get-LocalUser		
Name	Enabled	Description
----	-----	-----
Administrator	False	Built-in account for administering the computer/domain
DefaultAccount	False	A user account managed by the system.
DLarusso	True	High kick specialist.
Guest	False	Built-in account for guest access to the computer/domain
sshd	True	
WDAGUtilityAccount	False	A user account managed and used by the system for Windows Defender A...

`Get-LocalUser` will display the users on our host. These users only have access to this particular host. Let us say that we want to create a new local user named `JLawrence`. We can accomplish the task using `New-LocalUser`. If we are unsure of the proper syntax, please do not forget about the `Get-Help` Command. When creating a new local user, the only real requirement from a syntax perspective is to enter a `name` and specify a `password` (or `-NoPassword`). All other settings, such as a description or account expiration, are optional.

Creating A New User

User and Group Management		
PS C:\htb> New-LocalUser -Name "JLawrence" -NoPassword		
Name	Enabled	Description
----	-----	-----
JLawrence	True	

Above, we created the user `JLawrence` and did not set a password. So this account is active and can be logged in without a password. Depending on the version of Windows we are using, by not setting a Password, we are flagging to windows that this is a Microsoft live account, and it attempts to login in that manner instead of using a local password.

If we wish to modify a user, we could use the `Set-LocalUser` cmdlet. For this example, we will modify `JLawrence` and set a password and description on his account.

Modifying a User

User and Group Management		
PS C:\htb> \$Password = Read-Host -AsSecureString		

PS C:\htb> Set-LocalUser -Name "JLawrence" -Password \$Password -Description "CEO EagleFang"		
PS C:\htb> Get-LocalUser		
Name	Enabled	Description
----	-----	-----
Administrator	False	Built-in account for administering the computer/domain
DefaultAccount	False	A user account managed by the system.
demo	True	
Guest	False	Built-in account for guest access to the computer/domain
JLawrence	True	CEO EagleFang

As for making and modifying users, it is as simple as what we see above. Now, let us move on to checking out groups. If it feels like a bit of an echo...well, it is. The commands are similar in use.

Get-LocalGroup

User and Group Management

PS C:\htb> Get-LocalGroup

Name	Description
----	-----
Access Control Assistance Operators	Members of this group can remotely query authorization attr...
Administrators	Administrators have complete and unrestricted access to the...
Backup Operators	Backup Operators can override security restrictions for the...
Cryptographic Operators	Members are authorized to perform cryptographic operations.
Device Owners	Members of this group can change system-wide settings.
Distributed COM Users	Members are allowed to launch, activate and use Distributed...
Event Log Readers	Members of this group can read event logs from local machine
Guests	Guests have the same access as members of the Users group b...
Hyper-V Administrators	Members of this group have complete and unrestricted access...
IIS_IUSRS	Built-in group used by Internet Information Services.
Network Configuration Operators	Members in this group can have some administrative privileg...
Performance Log Users	Members of this group may schedule logging of performance c...
Performance Monitor Users	Members of this group can access performance counter data l...
Power Users	Power Users are included for backwards compatibility and po...
Remote Desktop Users	Members in this group are granted the right to logon remotely
Remote Management Users	Members of this group can access WMI resources over managem...
Replicator	Supports file replication in a domain
System Managed Accounts Group	Members of this group are managed by the system.
Users	Users are prevented from making accidental or intentional s...

PS C:\Windows\system32> Get-LocalGroupMember -Name "Users"

ObjectClass	Name	PrincipalSource
-----	-----	-----
User	DESKTOP-B3MFM77\demo	Local
User	DESKTOP-B3MFM77\JLawrence	Local
Group	NT AUTHORITY\Authenticated Users	Unknown
Group	NT AUTHORITY\INTERACTIVE	Unknown

In the output above, we ran the `Get-LocalGroup` cmdlet to get a printout of each group on the host. In the second command, we decided to inspect the `Users` group and see who is a member of said group. We did this with the `Get-LocalGroupMember` command. Now, if we wish to add another group or user to a group, we can use the `Add-LocalGroupMember` command. We will add `JLawrence` to the `Remote Desktop Users` group in the example below.

Adding a Member To a Group

User and Group Management

PS C:\htb> Add-LocalGroupMember -Group "Remote Desktop Users" -Member "JLawrence"

PS C:\htb> Get-LocalGroupMember -Name "Remote Desktop Users"

ObjectClass	Name	PrincipalSource
-----	-----	-----
User	DESKTOP-B3MFM77\JLawrence	Local

After running the command, we checked the group membership and saw that our user was indeed added to the Remote Desktop Users group. Maintaining local users and groups is simple and does not require external modules. Managing Active Directory Users and Groups requires a bit more work.

Managing Domain Users and Groups

Before we can access the cmdlets we need and work with Active Directory, we must install the **ActiveDirectory** PowerShell Module. If you installed the AdminToolbox, the AD module might already be on your host. If not, we can quickly grab the AD modules and get to work. One requirement is to have the optional feature **Remote System Administration Tools** installed. This feature is the only way to get the official ActiveDirectory PowerShell module. The edition in AdminToolbox and other Modules is repackaged, so use caution.

Installing RSAT

User and Group Management

```
PS C:\htb> Get-WindowsCapability -Name RSAT* -Online | Add-WindowsCapability -Online

Path          :
Online        : True
RestartNeeded : False
```

The above command will install **ALL** RSAT features in the Microsoft Catalog. If we wish to stay lightweight, we can install the package named **Rsat.ActiveDirectory.DS-LDS.Tools~~~~0.0.1.0**. Now we should have the ActiveDirectory module installed. Let us check.

Locating The AD Module

User and Group Management

```
PS C:\htb> Get-Module -Name ActiveDirectory -ListAvailable

Directory: C:\Windows\system32\WindowsPowerShell\v1.0\Modules

ModuleType Version      Name                               ExportedCommands
-----
Manifest    1.0.1.0      ActiveDirectory                   {Add-ADCentralAccessPolicyMember, Add-ADComputerService...
```

Nice. Now that we have the module, we can get started with AD **User** and **Group** management. The easiest way to locate a specific user is by searching with the **Get-ADUser** cmdlet.

Get-ADUser

User and Group Management

```
PS C:\htb> Get-ADUser -Filter *

DistinguishedName : CN=user14,CN=Users,DC=greenhorn,DC=corp
Enabled           : True
GivenName         :
Name              : user14
ObjectClass       : user
ObjectGUID        : bef9787d-2716-4dc9-8e8f-f8037a72c3d9
SamAccountName    : user14
SID               : S-1-5-21-1480833693-1324064541-2711030367-1110
Surname           :
UserPrincipalName :

DistinguishedName : CN=sshd,CN=Users,DC=greenhorn,DC=corp
Enabled           : True
GivenName         :
Name              : sshd
ObjectClass       : user
ObjectGUID        : 7a324e98-00e4-480b-8a1a-fa465d558063
SamAccountName    : sshd
SID               : S-1-5-21-1480833693-1324064541-2711030367-1112
```

```
Surname      :  
UserPrincipalName :  
  
DistinguishedName : CN=TSilver,CN=Users,DC=greenhorn,DC=corp  
Enabled        : True  
GivenName     :  
Name          : TSilver  
ObjectClass    : user  
ObjectGUID     : a19a6c8a-000a-4cbf-aa14-0c7fca643c37  
SamAccountName : TSilver  
SID           : S-1-5-21-1480833693-1324064541-2711030367-1602  
Surname       :  
UserPrincipalName :  
  
<SNIP>
```

The parameter `-Filter *` lets us grab all users within Active Directory. Depending on our organization's size, this could produce a ton of output. We can use the `-Identity` parameter to perform a more specific search for a user by `distinguished name`, `GUID`, `the objectId`, or `SamAccountName`. Do not worry if these options seem like gibberish to you; that is all right. The specifics of these are not important right now; for more reading on the topic, check out [this article](#) or the [Intro To Active Directory](#) module. We are going to search for the user `TSilver` now.

Get a Specific User

User and Group Management

```
PS C:\htb> Get-ADUser -Identity TSilver  
  
DistinguishedName : CN=TSilver,CN=Users,DC=greenhorn,DC=corp  
Enabled          : True  
GivenName       :  
Name            : TSilver  
ObjectClass     : user  
ObjectGUID      : a19a6c8a-000a-4cbf-aa14-0c7fca643c37  
SamAccountName  : TSilver  
SID             : S-1-5-21-1480833693-1324064541-2711030367-1602  
Surname         :  
UserPrincipalName :
```

We can see from the output several pieces of information about the user, including:

- **Object Class**: which specifies if the object is a user, computer, or another type of object.
- **DistinguishedName**: Specifies the object's relative path within the AD schema.
- **Enabled**: Tells us if the user is active and can log in.
- **SamAccountName**: The representation of the username used to log into the ActiveDirectory hosts.
- **ObjectGUID**: Is the unique identifier of the user object.

Users have many different attributes (not all shown here) and can all be used to identify and group them. We could also use these to filter specific attributes. For example, let us filter the user's `Email address`.

Searching On An Attribute

User and Group Management

```
PS C:\htb> Get-ADUser -Filter {EmailAddress -like '*greenhorn.corp'}  
  
DistinguishedName : CN=TSilver,CN=Users,DC=greenhorn,DC=corp
```



```
Enabled          : True
GivenName       :
Name            : TSilver
ObjectClass     : user
ObjectGUID      : a19a6c8a-000a-4cbf-aa14-0c7fca643c37
SamAccountName  : TSilver
SID             : S-1-5-21-1480833693-1324064541-2711030367-1602
Surname         :
UserPrincipalName :
```

In our output, we can see that we only had one result for a user with an email address matching our naming context

*[greenhorn.corp](#). This is just one example of attributes we can filter on. For a more detailed list, check out this [Technet Article](#), which covers the default and extended user object properties.

We need to create a new user for an employee named [Mori Tanaka](#) who just joined Greenhorn. Let us give the New-ADUser cmdlet a try.

New ADUser

```

User and Group Management

PS C:\htb> New-ADUser -Name "MTanaka" -Surname "Tanaka" -GivenName "Mori" -Office "Security" -OtherAttributes (
AccountPassword: *****
PS C:\htb> Get-ADUser -Identity MTanaka -Properties * | Format-Table Name,Enabled,GivenName,Surname,Title,Office,Mail

Name      Enabled GivenName Surname Title Office Mail
-----
MTanaka   True    Mori      Tanaka  Sensei Security MTanaka@greenhorn.corp
```

Ok, a lot is going on here. It may look daunting but let us dissect it. The [first](#) portion of the output above is creating our user:

- `New-ADUser -Name "MTanaka"` : We issue the `New-ADUser` command and set the user's `SamAccountName` to `MTanaka`.
- `-Surname "Tanaka" -GivenName "Mori"`: This portion sets our user's `Lastname` and `Firstname`.
- `-Office "Security"`: Sets the extended property of `Office` to `Security`.
- `-OtherAttributes @{ 'title'="Sensei"; 'mail'="MTanaka@greenhorn.corp" }`: Here we set other extended attributes such as `title` and `Email-Address`.
- `-Accountpassword (Read-Host -AsSecureString "AccountPassword")`: With this portion, we set the user's `password` by having the shell prompt us to enter a new password. (we can see it in the line below with the stars)
- `-Enabled $true`: We are enabling the account for use. The user could not log in if this was set to `\$False`.

The [second](#) is validating that the user we created and the properties we set exist:

- `Get-ADUser -Identity MTanaka -Properties *`: Here, we are searching for the user's properties `MTanaka`.
- `|` : This is the Pipe symbol. It will be explored more in another section, but for now, it takes our `output` from `Get-ADUser` and sends it into the following command.
- `Format-Table Name,Enabled,GivenName,Surname,Title,Office,Mail`: Here, we tell PowerShell to `Format` our results as a `table` including the default and extended properties listed.

Seeing the commands broken down like this helps demystify the strings. Now, what if we need to modify a user? `Set-ADUser` is our ticket. Many of the filters we looked at earlier apply here as well. We can change or set any of the attributes that were listed. For this example, let us add a `Description` to Mr. Tanaka

example, let us add a **description** to Mr. Tanaka.

Changing a Users Attributes

User and Group Management

```
PS C:\htb> Set-ADUser -Identity MTanaka -Description " Sensei to Security Analyst's Rocky, Colt, and Tum-Tum"

PS C:\htb> Get-ADUser -Identity MTanaka -Property Description

Description           :  Sensei to Security Analyst's Rocky, Colt, and Tum-Tum
DistinguishedName     :  CN=MTanaka,CN=Users,DC=greenhorn,DC=corp
Enabled                :  True
GivenName              :  Mori
Name                   :  MTanaka
ObjectClass            :  user
ObjectGUID             :  c19e402d-b002-4ca0-b5ac-59d416166b3a
SamAccountName         :  MTanaka
SID                    :  S-1-5-21-1480833693-1324064541-2711030367-1603
Surname                :  Tanaka
UserPrincipalName      :
```

Querying AD, we can see that the **description** we set has been added to the attributes of Mr. Tanaka. User and group management is a common task we may find ourselves doing as sysadmins. However, why should we care about it as a **pentester**?

Why is Enumerating Users & Groups Important?

Users and groups provide a wealth of opportunities regarding Pentesting a Windows environment. We will often see users misconfigured. They may be given excessive permissions, added to unnecessary groups, or have weak/no passwords set. Groups can be equally as valuable. Often groups will have nested membership, allowing users to gain privileges they may not need. These misconfigurations can be easily found and visualized with Tools like **Bloodhound**. For a detailed look at enumerating Users and Groups, check out the **Windows Privilege Escalation** module.

Moving On

Now that we have the User and Group management down let's move on to working with files, folders, and other objects with PowerShell.

VPN Servers

Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

us-academy-1

PROTOCOL

☒ UDP 1337 ☐ TCP 443

DOWNLOAD VPN CONNECTION FILE



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

79ms ▼

Terminate Pwnbox to switch location

Start Instance

∞ / 1 spawns left

Waiting to start...



Questions

Answer the question(s) below to complete this Section and earn cubes!

Target: [Click here to spawn the target system!](#)



Cheat Sheet

Download VPN
Connection File

+0 What resource can provide Windows environments with directory services to manage users, computers, and more? (full name not abbreviation)

Submit your answer here...

+10 Streak pts



Submit



Hint



Show Solution



+0 What PowerShell Cmdlet will display all LOCAL users on a host?

Submit your answer here...

+10 Streak pts



Submit



Hint



Show Solution



SSH to with user "**mtanaka**" and password "**HTB_@cademy_stdnt!**"

+ 0

Connect to the target host and search for a domain user with the given name of Robert. What is this users Surname?

Submit your answer here...

+10 Streak pts

Submit

Hint

Show Solution

Previous

Next

Cheat Sheet

Resources

Go to Questions

Table of Contents

Introduction



CMD

Command Prompt Basics



Getting Help



System Navigation



Working with Directories and Files



Gathering System Information



Finding Files and Directories



Environment Variables



Managing Services



Working With Scheduled Tasks



PowerShell

CMD Vs. PowerShell



All About Cmdlets and Modules



User and Group Management

Working with Files and Directories

Finding & Filtering Content

Working with Services

Working with the Registry

Working with the Windows Event Log

Networking Management from The CLI

Interacting with The Web

PowerShell Scripting and Automation

Finish Strong

Skills Assessment

 Skills Assessment

Beyond This Module

My Workstation

OFFLINE

 Start Instance / 1 spawns left